System Design

High Level Design

Introduction

- LLD
  - *LLD caters to developers and testers*
  - *internal logic of components, such as classes, methods, and algorithms, to enable accurate development and testing.*
  - *Testing: supporting unit and functional testing*
- HLD
  - *HLD aimed at architects and stakeholders*
  - *system's high-level structure, including modules, technologies, and interactions to guide architecture and integration planning.*
  - *Testing: integration and performance testing*

HLD

- a big picture of what the system is
- establishment of technical feasibility, resource allocation and development sequence.
- Typical Artifacts in HLD
  - *System Architecture Diagram*
  - *Technology Stack*
  - *Data Flow Diagrams (DFDs)*

HLD vs LLD

Aspect	High-Level Design (HLD)	Low-Level Design (LLD)
Purpose	Define system structure and module relationships	Define detailed module logic for development
Abstraction	High	Low
Focus	“What” the system does	“How” does the system do it
Audience	Architects, leads, and clients	Developers, testers
Documentation	Diagrams, module specs, tech stack	Class diagrams, logic flows, pseudo-code
Technology Involvement	Describes tools and platforms used	Applies those tools in design implementation
Output Usability	Drives architectural decisions and project planning	Drives code development and unit test creation
Detail Level	Conceptual, large-grained	Technical, fine-grained
Change Tolerance	More stable once defined	More flexible and iterative

HLD's Performance Goals

- **Response Time**: How quickly the application responds to user actions.
- **Throughput**: The number of requests the system can process in a given time frame.
- **Resource Utilization**: Monitoring CPU, memory, and network usage during the test.

Course Introduction

- CAP Theorem
  - *Consistency*
  - *Availability*
- Queue
- Load Balancer Algorithms
- Stateless vs Stateful API
- SQL vs NoSQL
- Scaling
  - *Server*
  - *Database*
- Database
  - *Indexing, Replication, Partitioning, Sharding*
- Caching

CAP Theorem

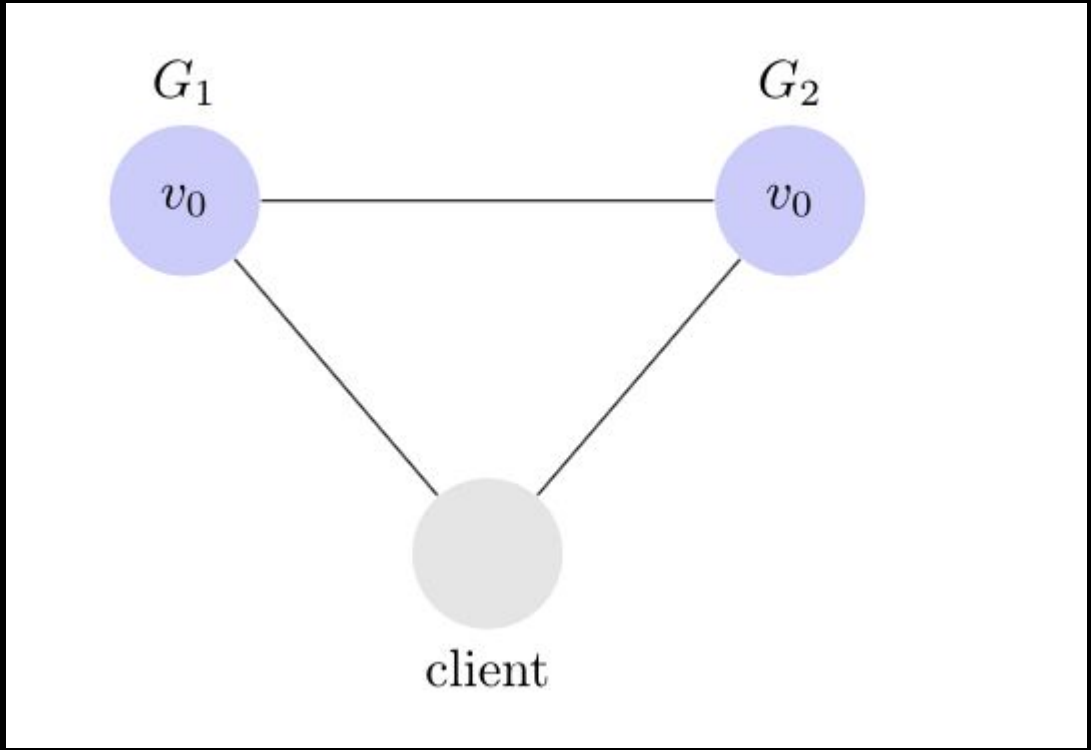
The CAP Theorem is a fundamental theorem in distributed systems that states any distributed system can have at most two of the following three properties:

- Consistency
- Availability
- Partition tolerance

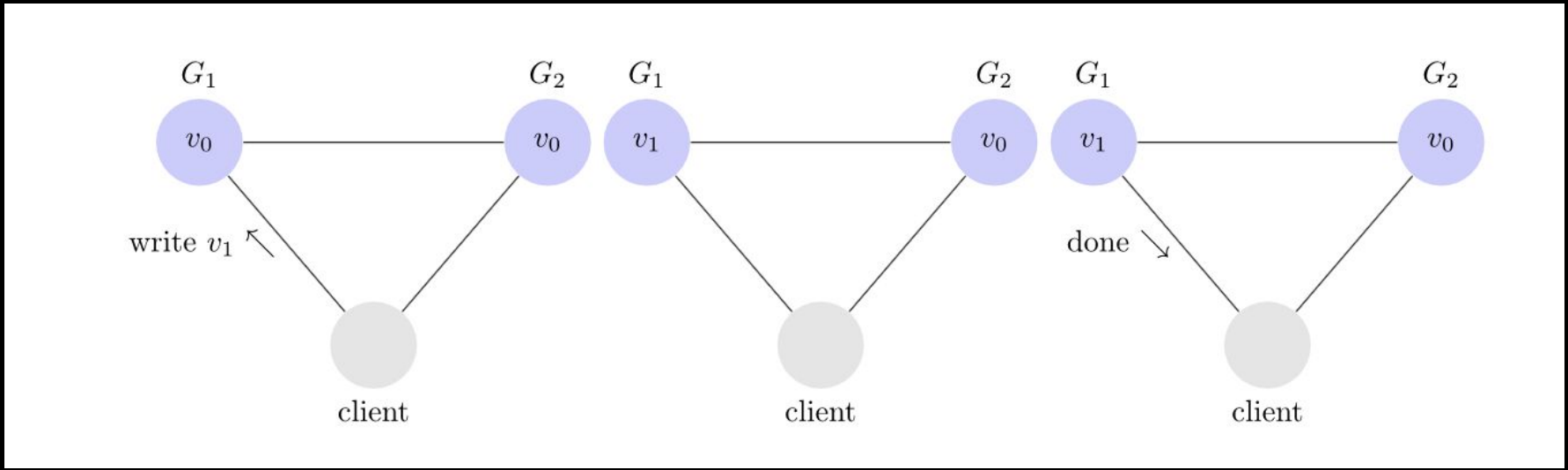
OR

The CAP theorem states that a distributed system cannot simultaneously be consistent, available, and partition tolerant.

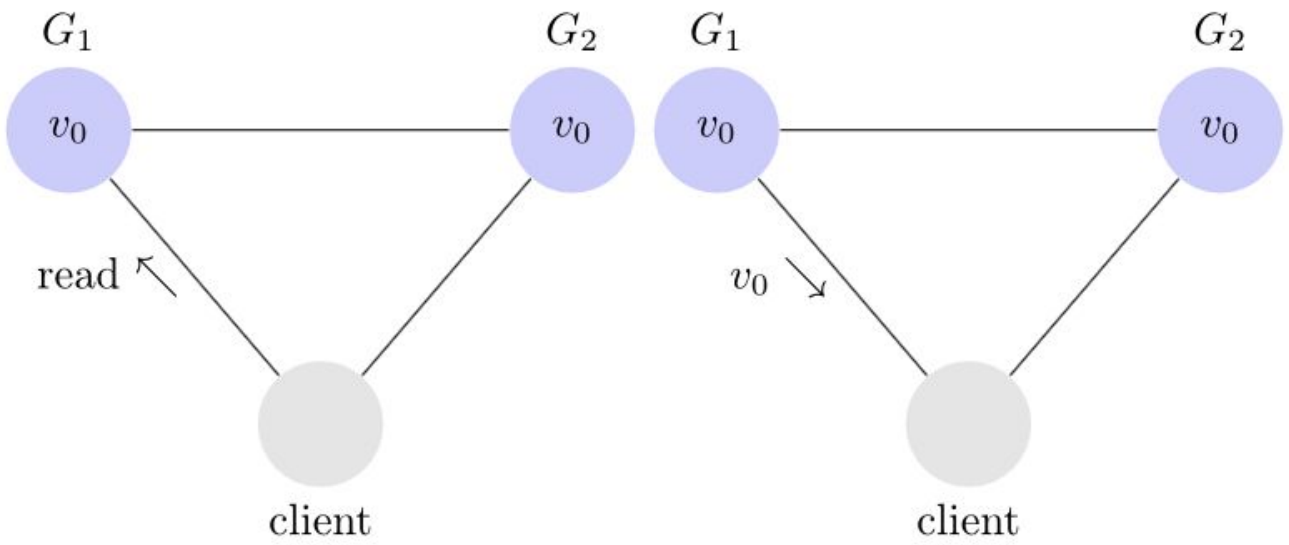
CAP Theorem: Simple Distributed System



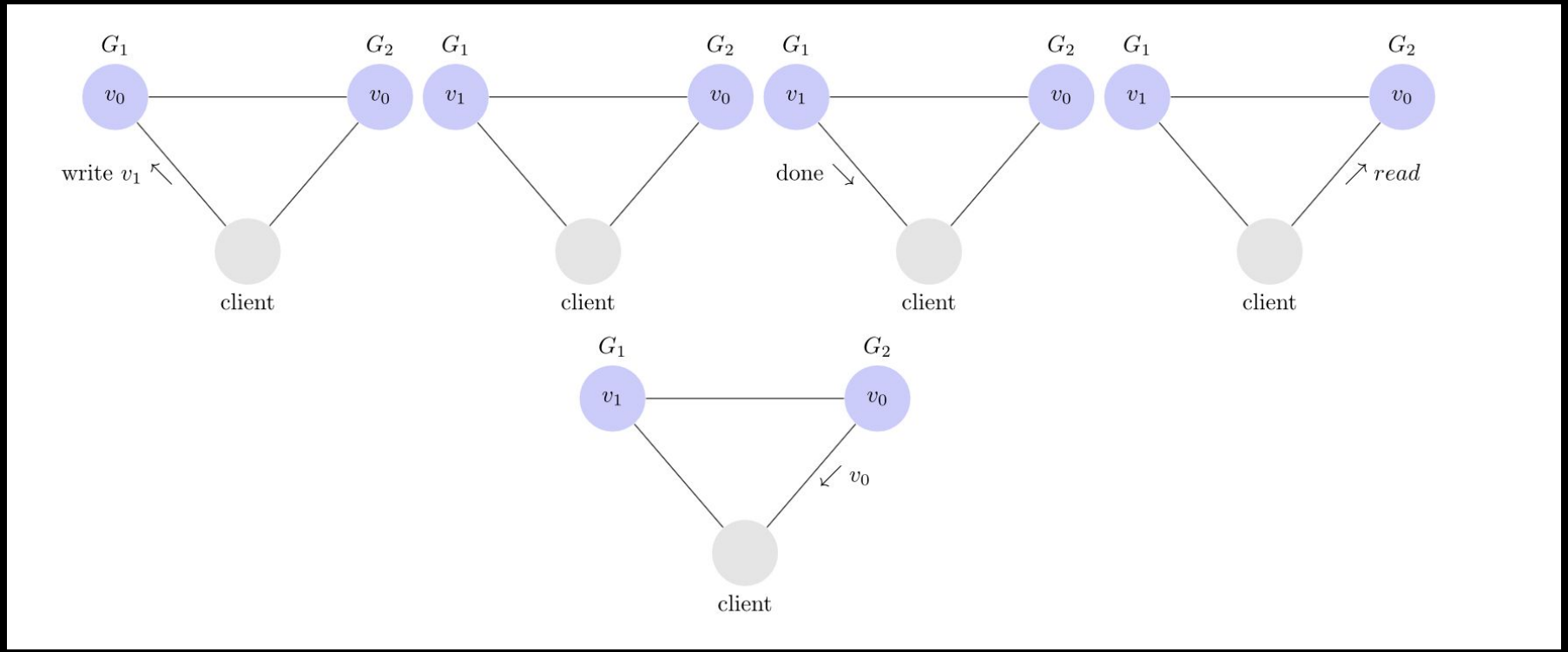
Distributed System: Write



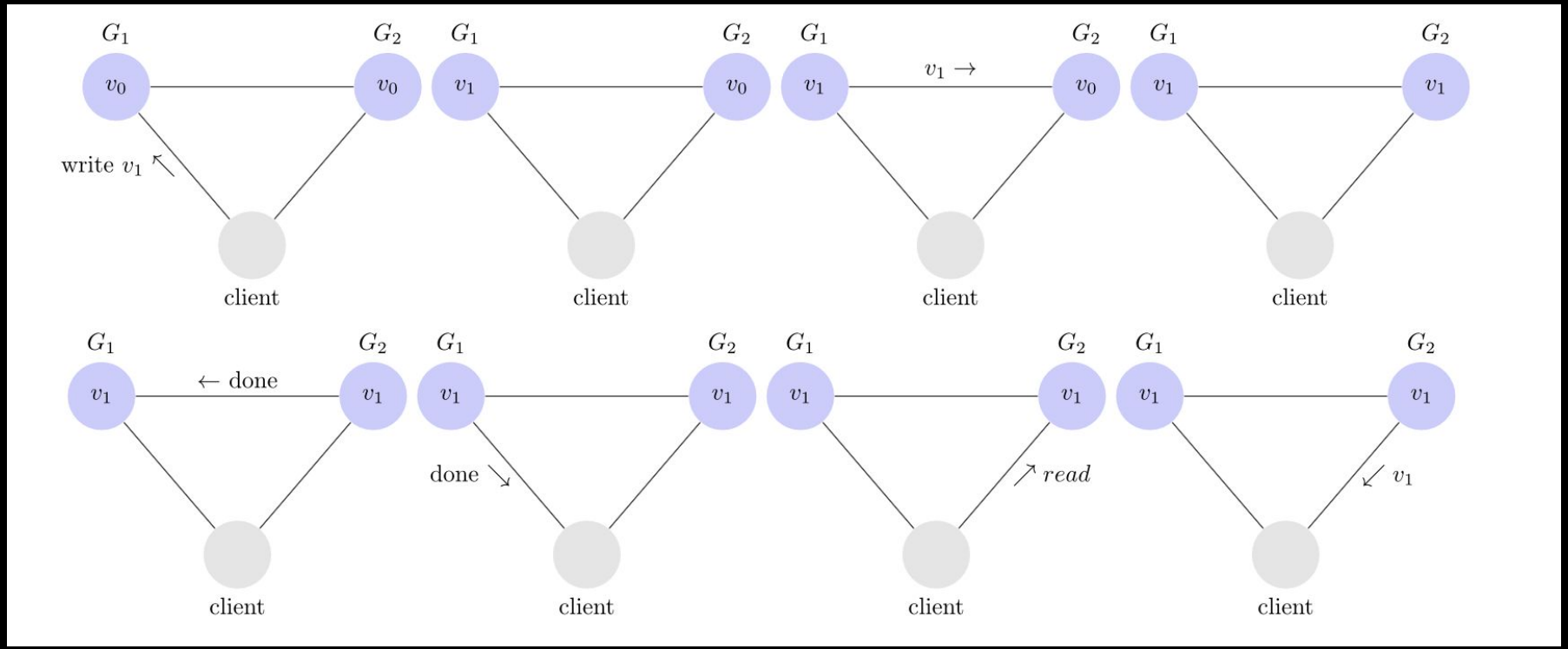
Distributed System: Read



Inconsistent Distributed System



Consistent Distributed System



Availability

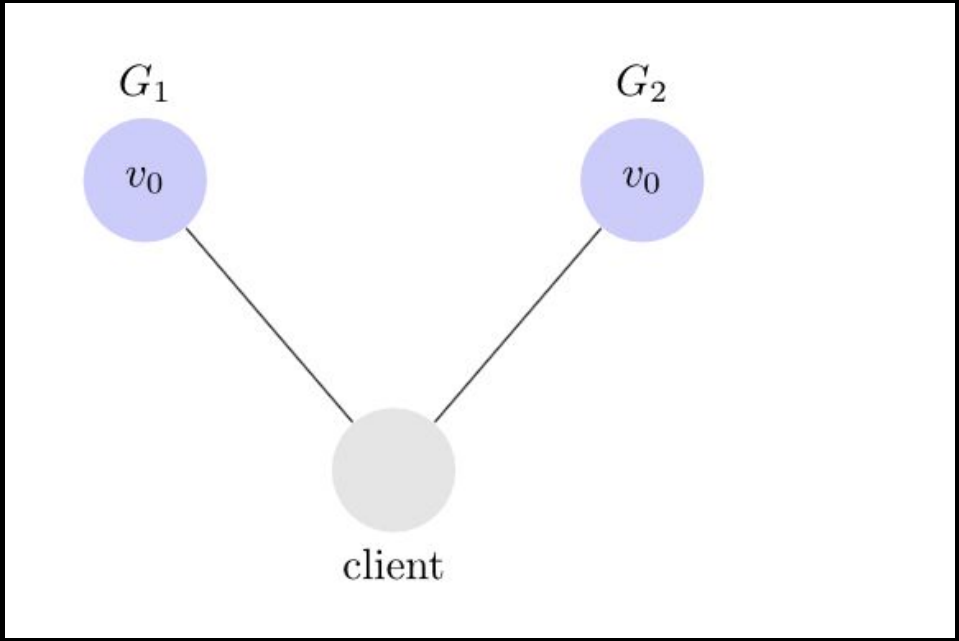
Every request received by a non-failing node in the system must result in a response

In an available system, if our client sends a request to a server and the server has not crashed, then the server must eventually respond to the client. The server is not allowed to ignore the client's requests.

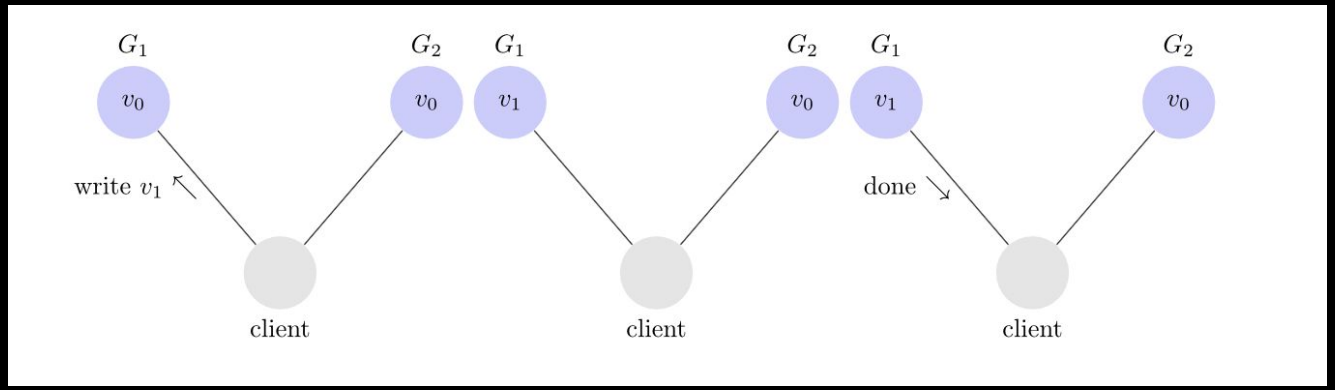
Partition Tolerance

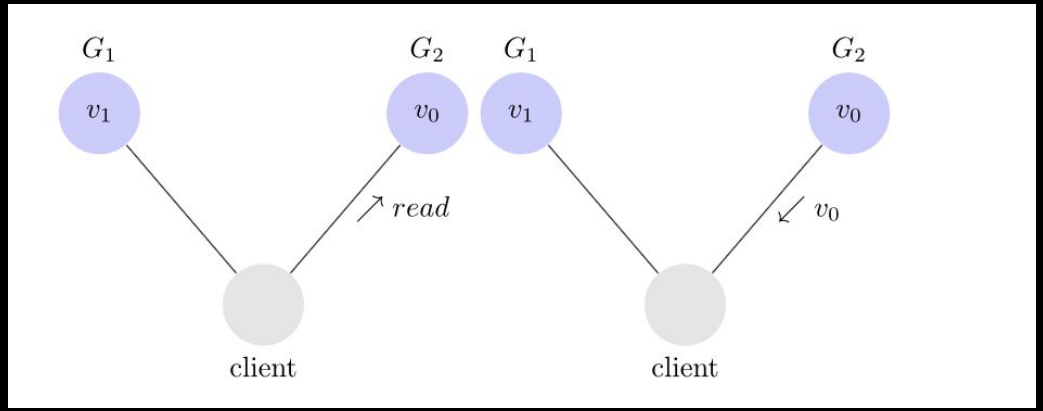
The network will be allowed to lose arbitrarily many messages sent from one node to another.

This means that any messages G1 and G2 send to one another can be dropped.



CAP Theorem: Proof





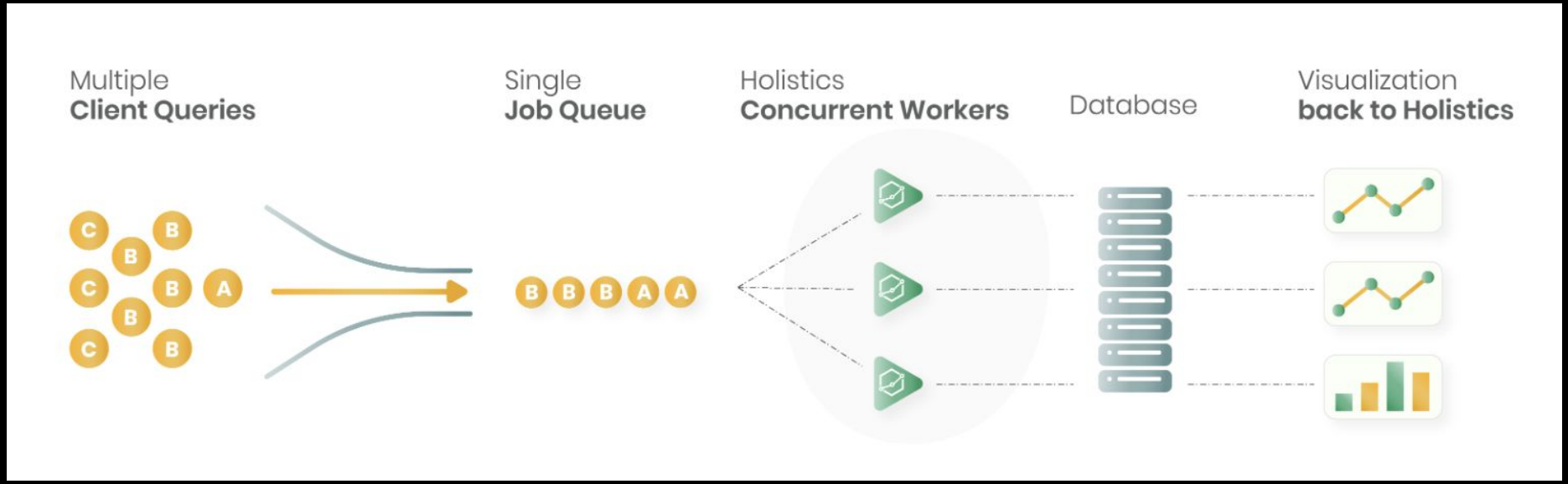
The Trade-off: CP vs. AP

Since Partition Tolerance (P) is necessary in distributed systems, the trade-off is actually between Consistency and Availability during a network fault

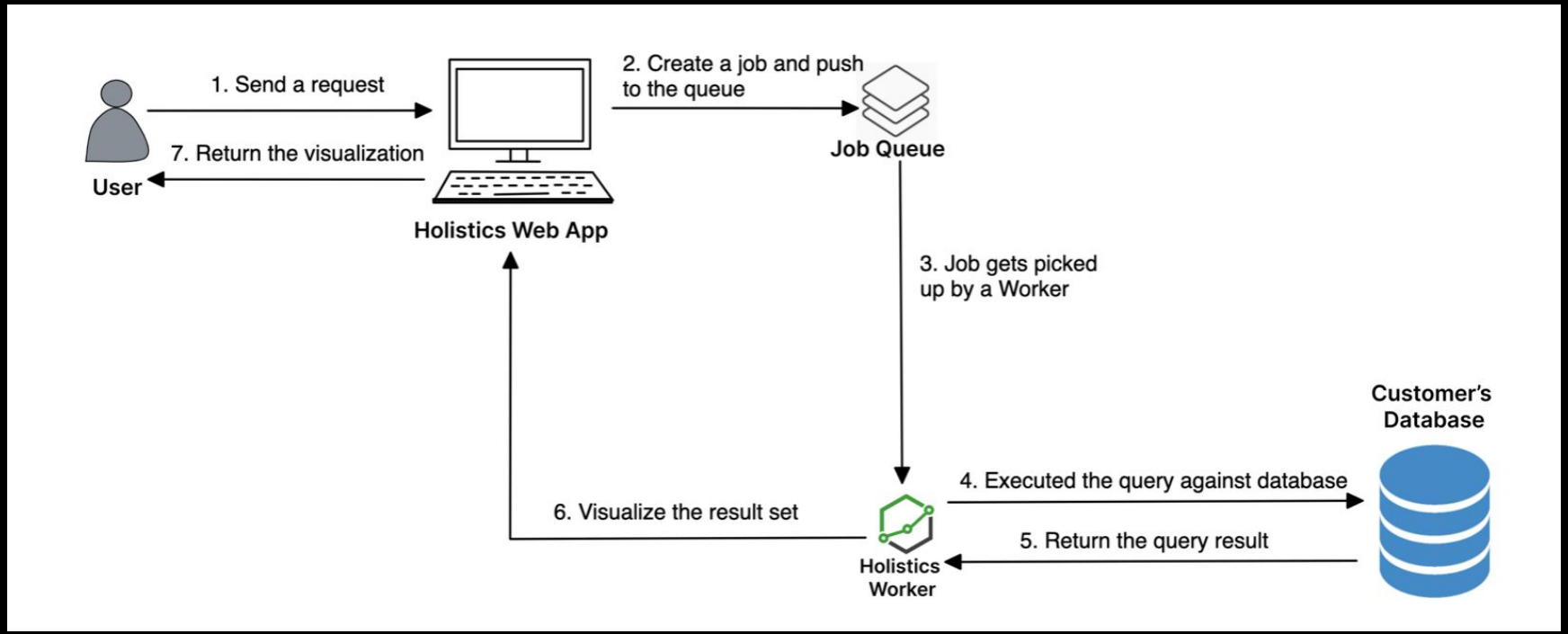
CP (Consistency + Partition Tolerance): The system prioritizes data accuracy. If a partition occurs, the system will return an error or time out rather than sharing inaccurate, stale data. (Example: Traditional relational databases like MySQL)

AP (Availability + Partition Tolerance): The system prioritizes being available. It will respond to requests with the best available data, even if it might be stale or inconsistent

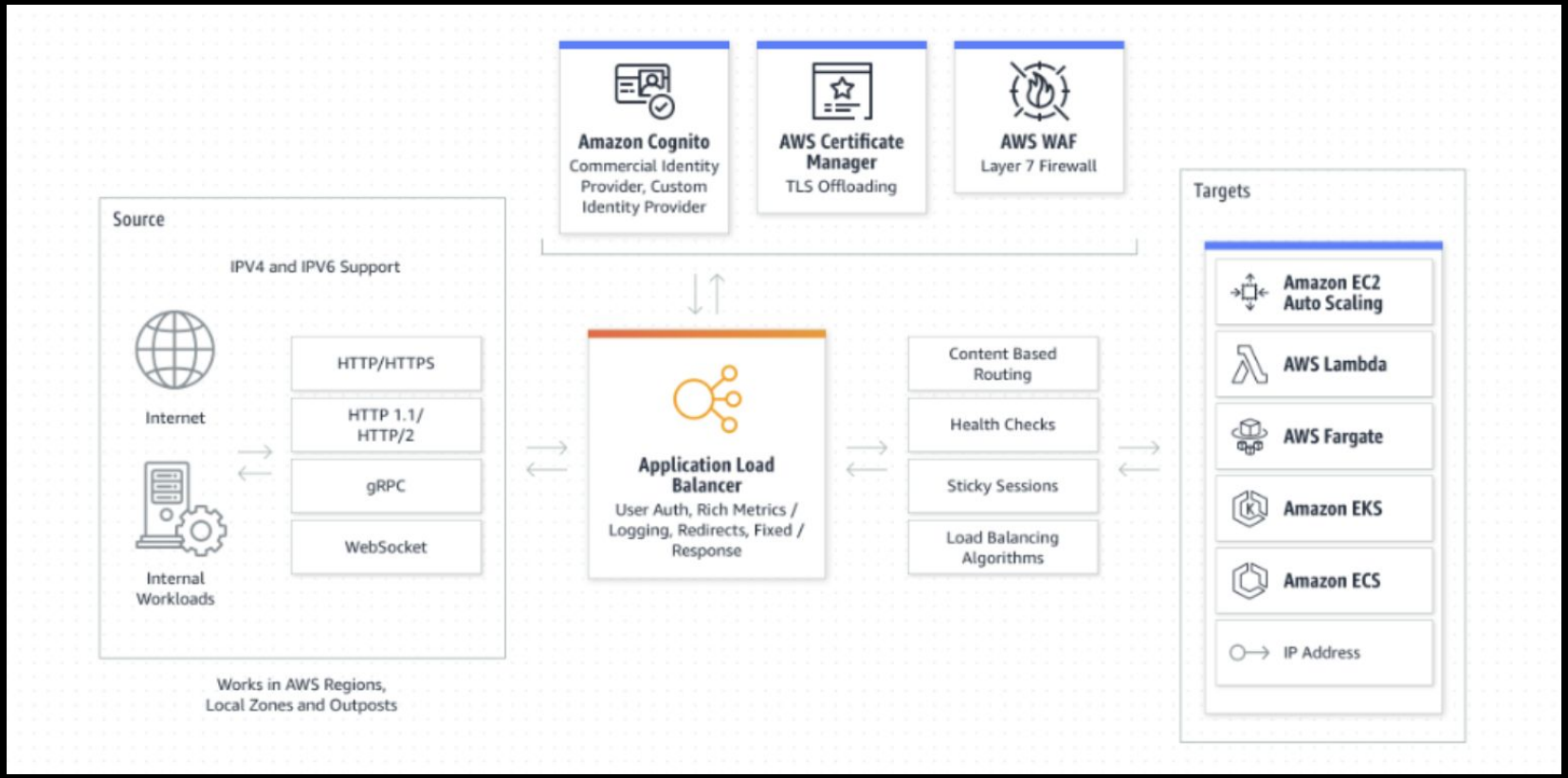
Queue


Multiple
Client Queries
Single
Job Queue
Holistics
Concurrent Workers
Database
Visualization
back to Holistics
C
B
C
B
B
A
B
C
B
B
B
B
A
A

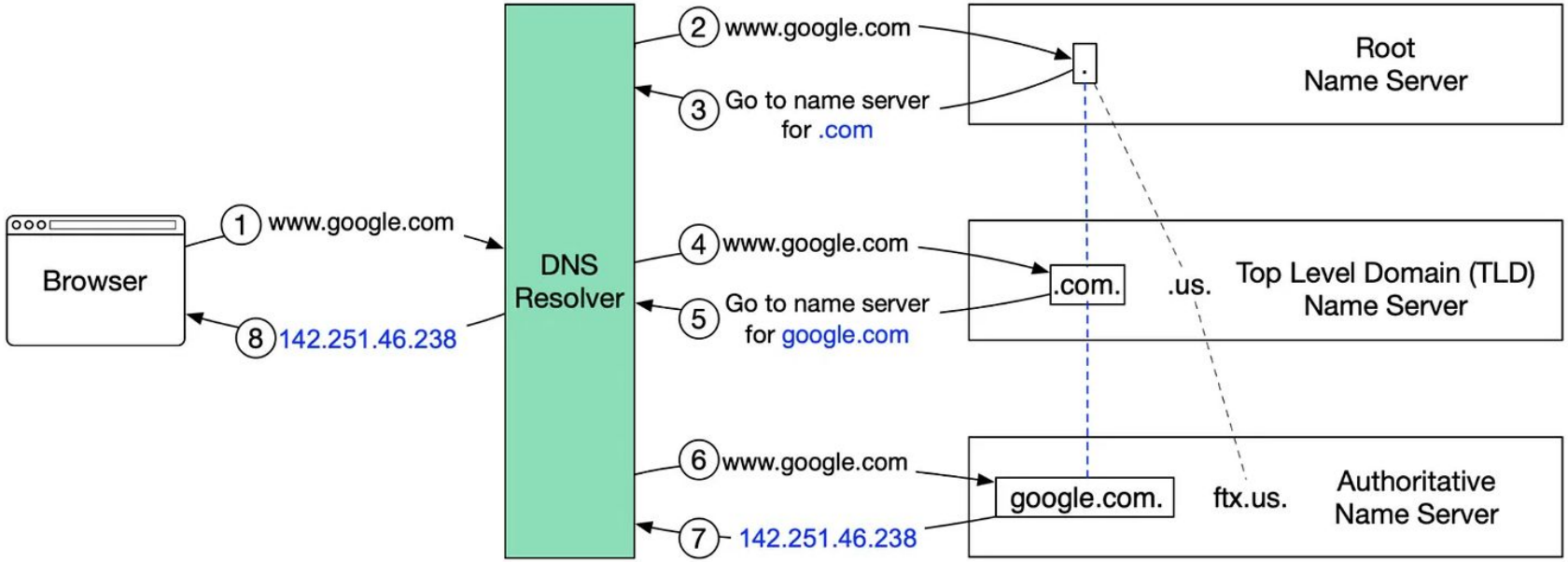
Queue


1. Send a request
2. Create a job and push to the queue
Job Queue
7. Return the visualization
User
Holistics Web App
3. Job gets picked up by a Worker
Customer's Database
4. Executed the query against database
6. Visualize the result set
5. Return the query result
Holistics Worker

Load Balancer



Load Balanced by Authoritative Name server



Stateless vs Stateful API

Feature	Stateless APIs	Stateful APIs
Session Storage	Client-side (tokens/cookies)	Server-side (databases/memory)
Scalability	High (independent requests)	Limited (session management)
Performance	Consistent, slower for real-time	Faster for session-based tasks
Use Cases	Distributed systems, RESTful APIs	Real-time apps, e-commerce

SQL vs NoSQL

SQL vs NoSQL

- SQL databases are
  - *relational,*
  - *schema-based systems*
  - *designed for strong consistency and,*
  - *structured data.*
- NoSQL databases are
  - *non-relational,*
  - *flexible schema systems*
  - *designed for scalability and*
  - *high throughput*

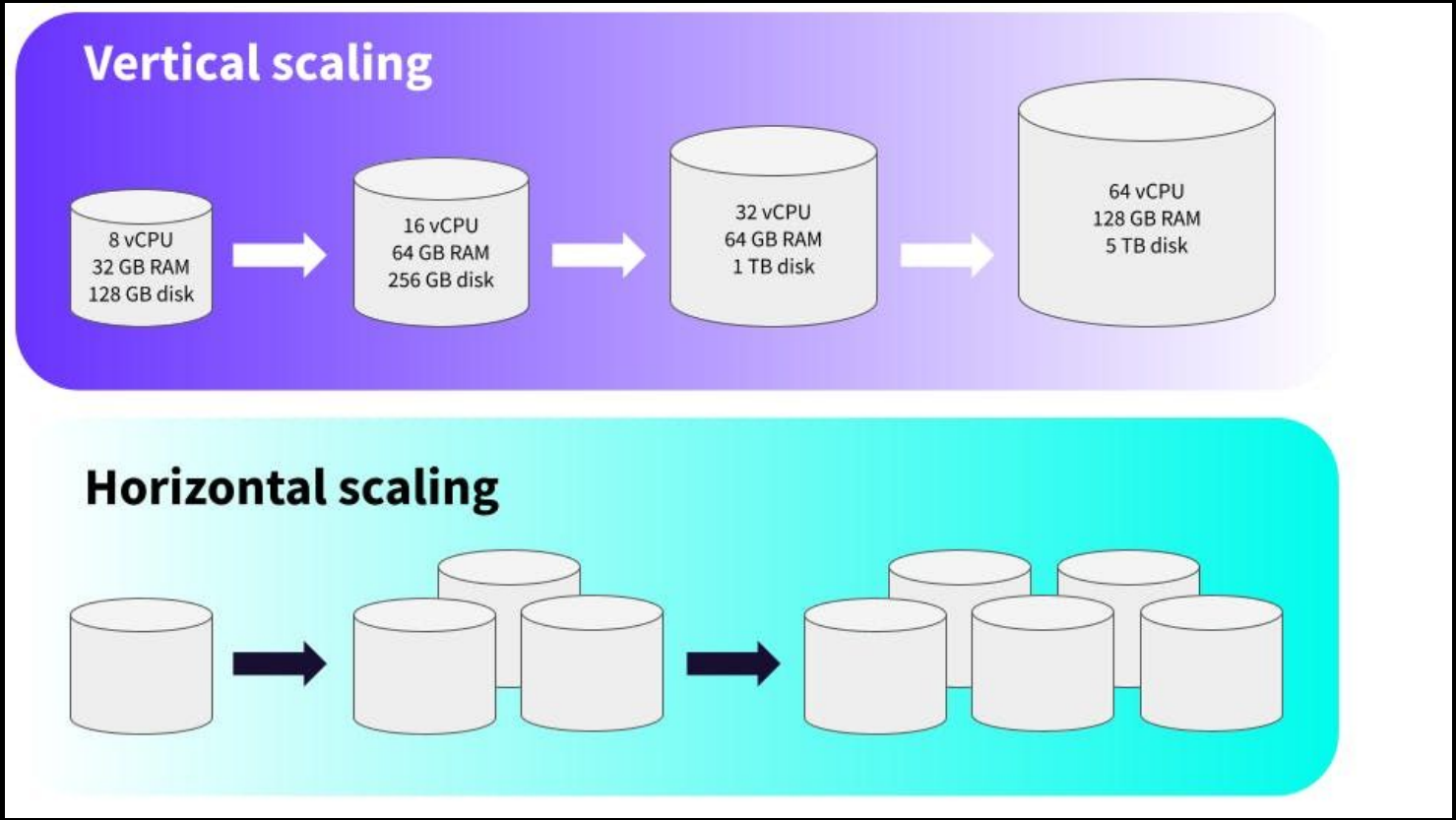
SQL vs NoSQL

> It’s not SQL vs NoSQL
> —it’s consistency vs
> Availability tradeoff

Interview Question

We want to scale a database to handle millions of users

DB Scaling Approaches



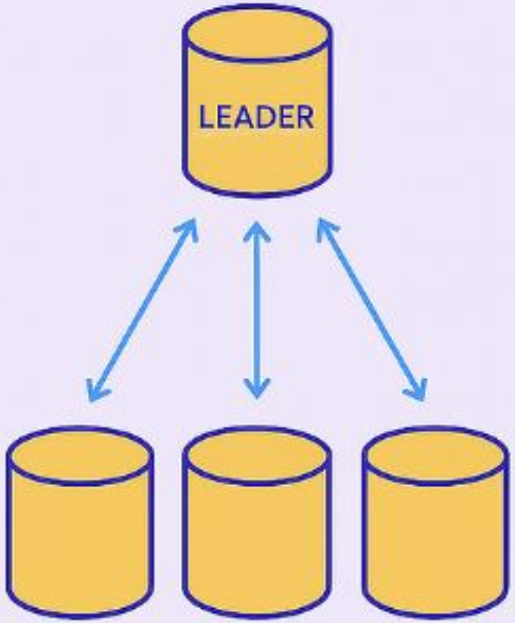
DB Scaling Approach

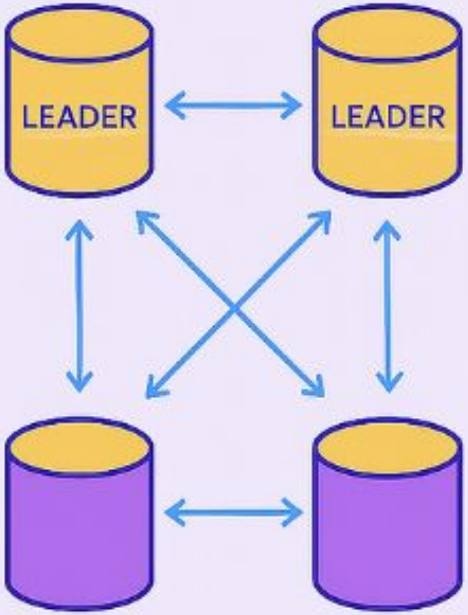
Full Name
Title

"I will prefer Horizontal DB Scaling"

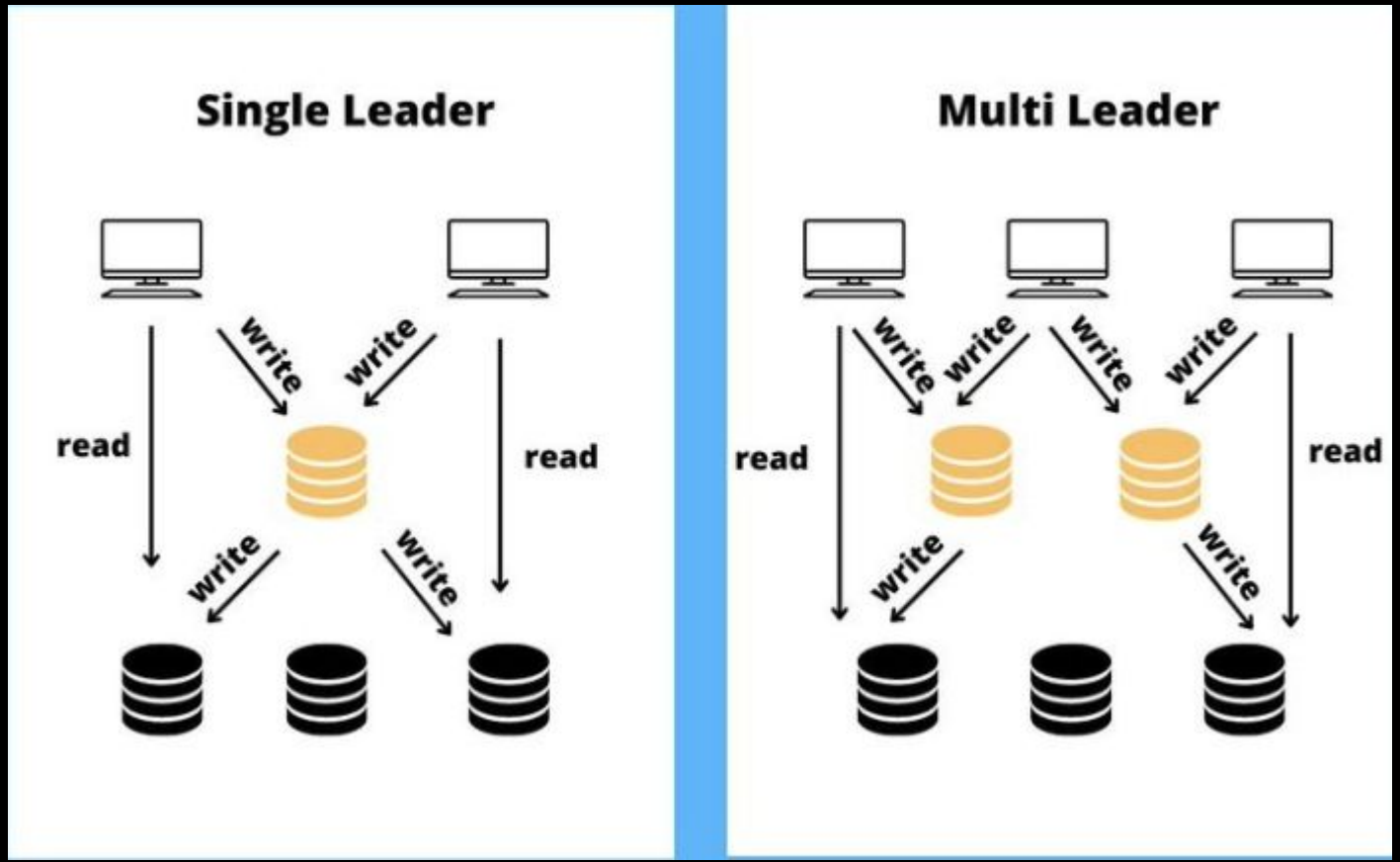
DB Replica

What if two nodes update same data differently?





Problem: How to manage Data?



Single Leader vs Multiple Leader

Single Leader

- Writes happen at one place
- No conflict
- System is simple and consistent
- But, Write scalability is limited

Multiple Leader

- Network delays
- Node failures
- Conflicting updates
- Scalability is high, Consistency is limited

SQL vs NoSQL: Traditional SQL

Traditional SQL (RDBMS)

- Example: MySQL, PostgreSQL
- Tend to favor:
  - *Consistency (C)*
  - *Over Availability during failures*

Behavior:

- If replicas disagree → system may reject requests
- Guarantees ACID

So:

- SQL → Consistency-first systems

SQL vs NoSQL

NoSQL Systems

- Example: MongoDB, Cassandra, DynamoDB
- Availability + Scalability is High
- Accept eventual consistency

Behavior:

- Always respond (even if slightly stale data)
- Sync happens later

So:

- NoSQL → Availability + Scalability-first systems

SQL vs NoSQL: Simple Answer

I choose SQL when I need **ACID guarantees** and complex joins, and NoSQL when I need **horizontal scalability**, flexible schema, or **high write/read throughput**

SQL: Banking, ERP

NoSQL: Social feeds, logs

SQL vs NoSQL: Detailed Answer

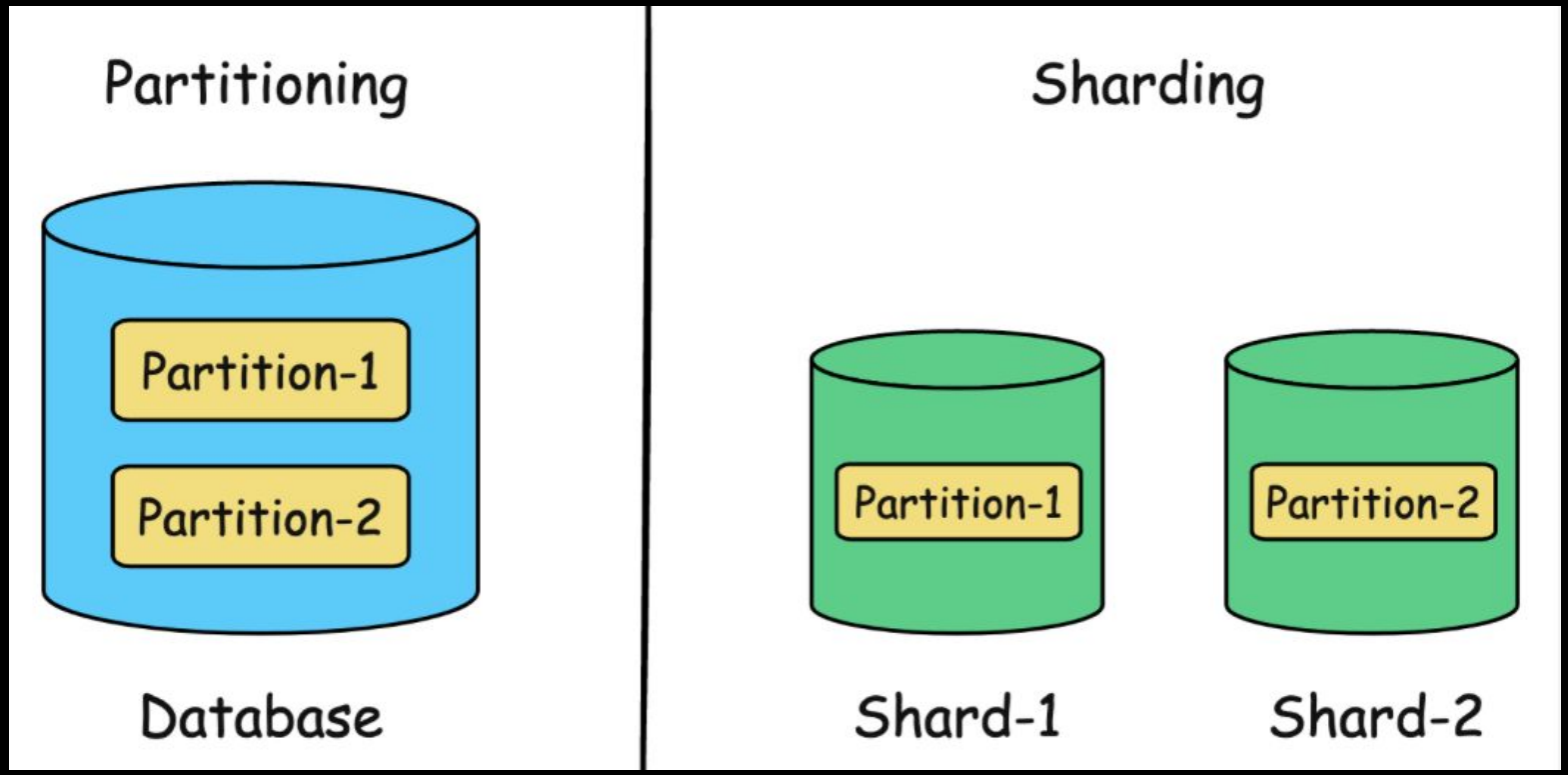
“The real tradeoff is not SQL vs NoSQL, but consistency vs availability under partition (CAP theorem).

SQL databases traditionally prioritize strong consistency using ACID guarantees, often sacrificing availability during failures.

NoSQL databases typically relax consistency (eventual consistency) to achieve higher scalability and availability.

However, modern databases blur this line by offering tunable consistency.”

Partitioning vs Sharding



URL Shortener